

Síntesis · Día 1 — El paradigma y la primera herramienta

La idea central del día

El taller no enseña a programar. Enseña a **dirigir** un programador que es la IA. El conocimiento matemático del profesor —qué confunde a los alumnos, qué rango de valores tiene sentido, cuál es el resultado correcto— es el activo que la máquina no tiene. Sin ese conocimiento, la IA genera código que puede correr perfecto y **mentir matemáticamente** (una escala mal puesta, una notación confundida). Por eso la validación matemática de todo lo que produce la IA es trabajo del profesor, y es irrenunciable.

A esto se le llama hacer **software de academia**: herramientas con una intención didáctica muy específica — el malentendido concreto de los alumnos— que ningún software comercial (GeoGebra, MATLAB) puede anticipar, porque solo el profesor conoce ese malentendido.

Los tres roles del profesor frente a la IA

- 1. **Arquitecto de la idea.** Antes de abrir la IA, se decide: qué muestra la herramienta, qué variables puede mover el alumno y en qué rango (con sentido matemático, no el rango por defecto que pondría la máquina).
- 2. **Validador matemático.** Se revisa que el resultado sea matemáticamente correcto. Que el programa corra sin errores **no significa nada** sobre si la matemática está bien.
- 3. **Iterador pedagógico.** Se decide qué mejorar para que el alumno entienda mejor, sin tocar el cálculo ya validado.

El flujo de trabajo completo

Idea matemática → Especificación → Prompt a la IA → Código → Ejecución → Validación → Iteración

Cómo se escribe un buen prompt: los cinco elementos

Un prompt no es una pregunta casual — es una especificación técnica. Si falta alguno de estos cinco elementos, la IA improvisa por su cuenta:

Elemento	Qué especifica	Qué inventa la IA si se omite
1. Contexto	Quiénes son los alumnos, qué saben	Un nivel arbitrario
2. Concepto exacto	Qué debe mostrar matemáticamente	Una interpretación genérica
3. Comportamiento	Qué debe hacer el programa	Una interfaz sin intención pedagógica

Elemento	Qué especifica	Qué inventa la IA si se omite
4. Restricciones	Plataforma, límites técnicos	Código que puede no correr donde se necesita
5. Validación	Cómo se sabrá que el resultado es correcto	Nada — y el error queda invisible

El **Elemento 5 es el que más se olvida y el único insustituible**: antes de ver el resultado, se calcula a mano cuál debe ser (por ejemplo: "para $n=4$, el área debe ser 18"). Si se calcula después de ver el resultado, el sesgo de confirmación hace que cualquier cosa parezca correcta.

Cómo leer el código que entrega la IA (sin saber programar)

Un programa típico en p5.js se divide en cinco "zonas". No es necesario memorizarlas, solo saber que existen y a cuál acudir según el síntoma:

Síntoma en pantalla	Zona a revisar
¿Qué función se está graficando?	Zona D — <code>function f(x)</code>
¿Cuál es el intervalo?	Zona A — <code>xMin, xMax</code>
¿Cuántos valores puede tomar el deslizador?	Zona C — números de <code>createSlider</code>
¿Por qué la curva se ve al revés?	Zona E — <code>mapY()</code>
¿Por qué la curva se sale de la pantalla?	Zona A — <code>yMin, yMax</code>

Regla práctica del día: si el cambio es un número o un color, se hace directamente (zonas A y B, riesgo bajo). Si es una línea completa o una función, se le pide a la IA (zona E, riesgo alto — no se toca a mano).

El "Error Tipo 0" — imprescindible

En p5.js el origen (0,0) está en la esquina **superior** izquierda y el eje Y crece **hacia abajo** — lo contrario del plano cartesiano de clase. Si no se le pide explícitamente a la IA, las gráficas salen invertidas. La frase que hay que exigir en el prompt: *"Invierte matemáticamente las coordenadas Y del lienzo para que el eje Y crezca hacia arriba, como en el plano cartesiano estándar."*

La plataforma: p5.js vs. Google Colab

Ninguna es superior a la otra; depende de qué se quiere mostrar:

- **Para algo que se mueve y responde** (geometría, animación, respuesta al clic) → **p5.js**. No requiere cuenta, abre en cualquier navegador, incluso el celular del alumno.
- **Para calcular con precisión o manejar datos** (estadística, cálculo simbólico) → **Google Colab / Python**. Requiere que el alumno tenga cuenta Google.

Orientación por área (no obligatoria): Cálculo, Álgebra lineal y EDO tienden a p5.js; Estadística y Álgebra discreta tienden a Colab; Geometría analítica tiende a p5.js. La elección del Día 1 es **individual y provisional** — la decisión definitiva por academia se toma el Día 5.

Producto del día

El ejercicio central fue: pedirle a la IA (Claude, ChatGPT, o similar) una herramienta interactiva de un concepto de la materia propia, con un deslizador que controle una variable relevante, siguiendo los cinco elementos anteriores. Al terminar, se validó en pareja contestando dos preguntas: *¿qué concepto muestra y qué no podía hacer el pizarrón?* y *¿cómo se sabe que es matemáticamente correcta?*

Tarea entre Día 1 y Día 2

Blueprint de Arquitectura Pedagógica (Doc13), con cuatro módulos:

- **A.** ¿Qué limitación del pizarrón resuelve la herramienta?
- **B.** Requerimientos: abstracción visual, variables manipulables y su rango
- **C.** Alertas del validador: Error Tipo 0 + al menos un caso de prueba calculado a mano
- **D.** Plan de prompts por capas: Capa 1 (núcleo matemático estático) → Capa 2 (interactividad) → Capa 3 (usabilidad)

Regla de entrada al Día 2: no se enciende ninguna computadora sin el Blueprint completo.

Términos clave del día

- **Director** — el papel del profesor frente a la IA: decide el rumbo, no escribe código.
- **Blueprint** — el documento de planeación pedagógica de la herramienta (se usa en inglés, sin traducir).
- **Software de academia** — herramientas hechas por y para el departamento, no software comercial.
- **Error Tipo 0** — la inversión del eje Y en p5.js frente al plano cartesiano.
- **Elemento 5** — el criterio de validación dentro de un prompt.